

UNIVERSIDADE FEDERAL DE ALAGOAS

INSTITUTO DE COMPUTAÇÃO

Gerar Commerce

Linha de produtos de software para e-commerce com microsserviços e componentes reutilizáveis

Tópico em Engenharia de Software
Projetando Linhas de Produto de Software

Caio Mascarenhas Soares
Laura Marques Costa

Repositório do projeto
https://github.com/CaioMascarenhas/gerar-commerce

Maceió
2026

Sumário

Repositório: https://github.com/CaioMascarenhas/gerar-commerce

1 Enunciado do projeto **4**

1.1 Domínio e produtos da família 4

2 Metodologia ágil escolhida **5**

3 Técnicas de reuso de software utilizadas **5**

3.1 Linha de produtos de software 5

3.2 Framework e componentes 6

3.3 Microsserviços 6

4 Desenvolvimento segundo a metodologia e as técnicas escolhidas **6**

4.1 Evidências e alcance da verificação 7

5 Product Backlog e Sprint Backlog **7**

5.1 Diagrama de features 11

5.2 Arquitetura em camadas e variabilidade 12

5.3 Componentes de software e variabilidade 14

5.4 Tela de configuração e construção de um produto 16

6 Arquitetura baseada em microsserviços **20**

6.1 Arquitetura e recomendações por histórico 20

6.2 Especificação documentada da API gateway 24

6.3 Especificação de cada microsserviço 26

6.4 Serviços em execução e respectivas portas 27

6.5 Passo a passo para construir outra aplicação 28

7 Tecnologias da implementação **29**

8 Funcionamento dos produtos por telas **31**

8.1 Plataforma e escolha de produto 31

8.2 Loja essencial e recomendações 32

8.3 Administração e fluxo de compra 33

8.4 Produto com inteligência artificial 34

8.5 Limites e evoluções 34

9 Repositório e instruções de execução **35**

9.1 Inicialização com Docker Compose 35

9.2 Configuração opcional de Gemini 36

9.3 Desenvolvimento e verificações 36

10 Referências bibliográficas **36**

1 Enunciado do projeto

O Gerar Commerce implementa uma linha de produtos de software para lojas de e-commerce. A proposta é produzir aplicações de um mesmo domínio a partir de ativos comuns e pontos de variação explícitos. Cada lojista cadastra uma identidade, escolhe uma categoria e seleciona funcionalidades opcionais. A plataforma monta uma loja com endereço próprio, sem duplicar a aplicação web ou os microsserviços.

A família compartilha catálogo, estoque, carrinho, checkout, pedidos, recomendações e administração. Esses sete recursos são obrigatórios. Cupons, avaliações, entrega local e agente de inteligência artificial são opcionais independentes. Uma categoria entre Eletrônicos, Construção e Roupas determina o tema visual. Nome, descrição, capa e produtos particularizam cada instância.

O cadastro exige pelo menos um produto e publica a vitrine em `/loja/:slug`, com painel em `/admin/:slug`. A compra exige conta de comprador da loja e utiliza pagamento simulado. O lojista administra a loja com senha própria. O sistema persiste dados no PostgreSQL e é implantado com Docker Compose, Angular no frontend e NestJS no backend.

Este relatório segue os itens do modelo de Linha de Produtos de Software fornecido pela disciplina [1]. Sua referência de implementação é o commit `5435ce33c7c0cb0f5cd226938793e771464b64a9`, da branch principal do GitHub, que incorpora as pull requests 47 e 48. Essa revisão contém o microsserviço de recomendações restaurado e os diagramas de camadas e variabilidade. O código completo é referenciado por links; as listagens do documento se concentram nos mecanismos que explicam a derivação e o funcionamento dos produtos.

1.1 Domínio e produtos da família

Configuração demonstrada	Categoria e variantes	Plano fictício
Aurora Eletrônicos	Eletrônicos; somente os sete recursos obrigatórios	Essencial, R$ 59,90/mês
Casa Nova Construção	Construção; núcleo com cupons e entrega local	Completo, R$ 79,90/mês
Ateliê Horizonte	Roupas; núcleo com cupons, avaliações e IA	Inteligente, R$ 99,90/mês

As lojas utilizam a mesma base de código e diferem por dados de configuração. O plano é calculado no servidor: nenhum opcional resulta em Essencial; opcionais sem IA resultam em Completo; selecionar IA resulta em Inteligente. O último preço não seleciona automaticamente os demais opcionais. Valores, vendas e pagamentos são fictícios, sem contratação ou cobrança financeira real.

2 Metodologia ágil escolhida

Foi adotado Kanban com o fluxo Backlog, Pronto, Em andamento, Em revisão e Concluído. Os itens possuem identificadores PB, prioridades, dependências e critérios de aceite. A definição de início exige entendimento da tarefa e de suas dependências. A conclusão exige implementação aplicável, verificação, documentação coerente e evidência. O limite sugerido é de dois itens em andamento e dois em revisão.

O método permite incorporar refinamentos sem perder o histórico. O projeto passou de perfis estáticos para cadastro dinâmico e recebeu contas de compradores, cupons próprios e ajustes de experiência. A visualização e as políticas de fluxo se relacionam às práticas descritas no Kanban Guide [2]. O repositório não possui uma série suficiente de métricas de fluxo para apresentar tempo médio de ciclo ou vazão como resultados medidos.

Para atender ao modelo acadêmico, foram registrados Product Backlog e snapshots chamados Sprint Backlog. Eles representam conjuntos selecionados em ciclos do Kanban. O relatório não pressupõe papéis ou cerimônias de Scrum que não foram adotados.

Ciclo planejado	Período em novembro de 2026	Meta
I1	04 a 05	Fundação, framework, perfis e implantação
I2	06 a 08	Persistência, compra e administração
I3	09 a 10	Experiência, IA, recomendações, verificação e relatórios

Issues, pull requests e snapshots permitem relacionar trabalho e entrega. Os commits usam Conventional Commits [3]. O plano inicial foi refinado e parte do trabalho de I2 foi antecipada para 05/11; as datas planejadas não são apresentadas como datas de execução de cada tarefa.

3 Técnicas de reuso de software utilizadas

3.1 Linha de produtos de software

A abordagem de LPS organiza sistemas relacionados em torno de funcionalidades e ativos comuns [4]. No projeto, a engenharia de domínio identifica o núcleo, os contratos, os pontos de variação e as implementações disponíveis. A engenharia de aplicação seleciona variantes e fornece identidade e catálogo para produzir uma loja concreta.

O vínculo das variantes ocorre em tempo de execução, no cadastro e nas configurações. O resultado é uma família multi-inquilino: lojas coexistem na mesma implantação e suas operações são delimitadas por `store_slug`. O isolamento é lógico; uma loja não recebe contêiner ou banco físico exclusivo. Há reuso de execução dos serviços, além do reuso de código.

3.2 Framework e componentes

`@gerar/framework` reúne composição e políticas. A classe `CommerceApplication` define um Template Method em `build()`: obter configuração, validar, registrar núcleo, registrar extensões, conferir componentes e calcular entrega. `ConfiguredStore` adapta esses passos a uma definição de loja. `ComponentRegistry` associa features a componentes e serviços. Essa organização preserva o fluxo comum e concentra a variação nos hotspots [5].

`ShippingPolicy`, `DiscountPolicy` e `PaymentProvider` separam políticas reutilizáveis. Retirada gratuita e entrega fixa, desconto percentual de cupom e pagamento simulado atendem contratos estáveis. Os componentes Angular, como `FeaturePicker`, `ImageEditor`, `BuyerAccount` e `ShoppingChat`, são reutilizados nas instâncias da família.

Os serviços importam o framework como pacote de workspace, e não por chamada HTTP. Lojas usa `composeStore()`; pedidos usa `quoteCheckout()` e `SimulatedPayment`; IA reutiliza as categorias. Estoque e catálogo executam componentes da arquitetura e usam os auxiliares de `platform`, mas não importam diretamente as regras do framework em seus controladores. A associação por descritores não significa que o framework instancie os controladores NestJS.

3.3 Microsserviços

As capacidades de negócio são distribuídas por processos HTTP/JSON. Lojas, catálogo, estoque, pedidos, recomendações e IA são acessados por um gateway. A decomposição por capacidade de negócio e os limites de responsabilidade seguem a perspectiva de arquitetura de microsserviços [6]. `contracts` compartilha tipos e `platform` compartilha inicialização, HTTP, validação e acesso ao banco.

O serviço de recomendações é independente do chat Gemini. O primeiro executa um ranking numérico com dados de compras e estoque, sem modelo externo. O segundo usa linguagem natural para auxiliar o comprador com catálogo e disponibilidade. A seleção de IA não controla a existência das recomendações, pois elas pertencem ao núcleo.

4 Desenvolvimento segundo a metodologia e as técnicas escolhidas

O incremento de 04/11 estruturou repositório, gestão, modelagem, contratos e framework, com perfis Essencial e Assistida. Em 05/11, a ADR-009 substituiu um perfil por implantação pela composição dinâmica. Foram antecipados persistência, onboarding, administração, checkout e parte dos opcionais. Os snapshots preservam essa mudança em vez de reconstruir uma sequência fictícia de entregas.

Em 09/11, contas e sessões de compradores passaram a pertencer à loja. Cupons cadastrados, imagens ajustáveis, chat público e consultas administrativas com Gemini ampliaram a família. As verificações do provedor utilizaram dados fictícios. Em 10/11, testes em navegador identificaram e corrigiram falhas de checkout, confirmação e foco de diálogos.

A revisão PB-043 retirou temporariamente as recomendações, além de corrigir navegação e avaliações. A ADR-011 e a pull request 47 restauraram esse componente como requisito obrigatório, agora com ranking por categorias, cestas, popularidade e preço. A pull request 48 incorporou diagramas de camadas, componentes e variabilidade. O histórico de retirada permanece registrado, mas o escopo deste relatório corresponde à restauração.

Os nomes da equipe foram conferidos no README e no backlog da revisão do GitHub. O histórico recente identifica a participação de Laura na integração das recomendações e dos diagramas. O relatório não atribui outras atividades individuais além do que os registros permitem verificar.

4.1 Evidências e alcance da verificação

Verificação desta revisão	Resultado registrado
Builds de backend e frontend	Concluídos pelo runner Docker antes das capturas.
Cadastro pela interface	Ateliê Horizonte publicado com 7 recursos core, 3 opcionais e plano de 9990 centavos.
Ranking personalizado	Hub Beta em primeiro; produto comprado e produto esgotado excluídos.
Visitante	Estratégia store-popularity, com Sensor Alpha em primeiro.
Isolamento de histórico	Consulta a ID diferente da conta retorna 403.
Saúde	Seis serviços internos e gateway responderam com status ok.
Gemini nesta rodada	Desativado; nenhuma nova chamada real foi declarada.

As verificações anteriores de 09 e 10/11 registram testes de frontend, backend, PostgreSQL, navegadores e Gemini. Elas pertencem às revisões indicadas em seus arquivos. A rodada própria deste relatório usa a revisão com recomendações restauradas e verifica cadastro, composição, histórico, ranking, isolamento e saúde dos serviços com dados fictícios. Não se apresenta a rodada de capturas como uma nova execução integral de todos os testes históricos.

5 Product Backlog e Sprint Backlog

O Product Backlog local contém 43 itens. P0, P1 e P2 são prioridades registradas, sem estimativas de esforço. Os estados distinguem implementação, revisão e aceite. PB-021 corresponde a este relatório e passa a Em revisão com sua produção. O quadro local é acompanhado de snapshots; isso não representa sincronização automática do Project do GitHub.

ID	Item do Product Backlog	P	Ciclo	Estado registrado
PB-001	Preparar monorepo, repositório, Kanban e backlogs	P0	I1	Concluído

ID	Item do Product Backlog	P	Ciclo	Estado registrado
PB-002	Modelar arquitetura, features e rastreabilidade acadêmica	P0	I1	Concluído
PB-003	Implementar framework com Template Method e hotspots	P0	I1	Concluído
PB-004	Derivar duas lojas pelo mesmo framework	P0	I1	Concluído
PB-005	Disponibilizar microsserviços de demonstração e gateway	P0	I1	Concluído
PB-006	Criar interface Angular para explorar loja e composição	P0	I1	Concluído
PB-007	Configurar CI com build, testes e smoke de integração	P0	I1	Concluído
PB-008	Persistir os domínios em PostgreSQL com isolamento	P0	I2	Concluído
PB-009	Implementar manutenção simples do catálogo	P0	I2	Concluído
PB-010	Implementar reserva e liberação de estoque	P0	I2	Concluído
PB-011	Implementar carrinho e resumo da compra no Angular	P0	I2	Concluído
PB-012	Implementar checkout com pagamento simulado e políticas do framework	P0	I2	Concluído
PB-013	Consultar pedidos, estados e resumo administrativo	P0	I2	Concluído
PB-014	Recomendar com histórico real das compras de demonstração	P0	I2	Concluído
PB-015	Criar lojas pela interface com composição dinâmica e opcionais	P0	I3	Concluído
PB-016	Validar assistente Mastra + Gemini com estoque e vendas	P0	I3	Concluído
PB-017	Administrar lojas com login simples por senha	P1	I2	Concluído
PB-018	Documentar contratos HTTP e APIs de cada serviço	P0	I3	Pronto
PB-019	Validar as duas aplicações de ponta a ponta	P0	I3	Em revisão
PB-020	Validar Docker Compose e capturar serviços com portas	P0	I1	Concluído
PB-021	Redigir relatório da LPS e reunir evidências	P0	I3	Em revisão
PB-022	Redigir relatório de reuso e documentar duas aplicações	P0	I3	Em revisão
PB-023	Congelar demonstração e ensaiar apresentação de 11/11	P0	I3	Pronto
PB-024	Adicionar cupons como componente opcional	P1	I2	Concluído
PB-025	Adicionar variações de produto como feature opcional	P2	Futuro	Backlog
PB-026	Adicionar avaliações de produtos como feature opcional	P1	I2	Concluído
PB-027	Separar frontend e backend com dependências independentes	P0	I1	Concluído

ID	Item do Product Backlog	P	Ciclo	Estado registrado
PB-028	Compor e persistir lojas dinâmicas por endereço	P0	I2	Concluído
PB-029	Criar onboarding, rotas de loja e painel administrativo	P0	I2	Em revisão
PB-030	Aplicar temas de categoria e três planos fictícios	P0	I2	Concluído
PB-031	Gerar três produtos para agilizar o cadastro	P0	I2	Concluído
PB-032	Testar isolamento, persistência e retentativas de compra	P0	I2	Concluído
PB-033	Criar contas de compradores e exigir login no checkout	P0	I3	Concluído
PB-034	Cadastrar e administrar cupons próprios da loja	P0	I3	Em revisão
PB-035	Disponibilizar chat Gemini e links individuais de produtos	P0	I3	Em revisão
PB-036	Criar sacola flutuante animada e acesso à própria loja	P0	I3	Em revisão
PB-037	Adicionar fotos e ajuste de enquadramento de capas e produtos	P0	I3	Em revisão
PB-038	Validar contas, cupons e experiência ampliada	P0	I3	Concluído
PB-039	Identificar perfis conectados e permitir logout independente	P0	I3	Concluído
PB-040	Corrigir sobreposição e foco do login ao finalizar a compra	P0	I3	Concluído
PB-041	Padronizar tipografia com Inter nos textos e Exo nos títulos	P1	I3	Concluído
PB-042	Criar ícones originais para as três categorias de loja	P1	I3	Concluído
PB-043	Corrigir navegação da loja e avaliações verificadas e retirar recomendações	P0	I3	Concluído

PB-014 registra a implementação histórica de recomendações e PB-043 registra sua retirada temporária. A restauração está comprovada pelos commits da pull request 47, sem criação retroativa de um item PB inexistente na base. PB-025 permanece evolução futura. O relatório de Reuso, PB-022, é uma entrega separada e mantém sua autoria individual.

Sprint Backlog I1

Foram selecionados PB-001 a PB-007 e PB-020; PB-027 refinou a separação de frontend e backend. A meta era tornar o reuso inspecionável e a infraestrutura executável. As entregas registradas abrangem repositório, contratos, Template Method, dois perfis, interface inicial, CI e Docker.

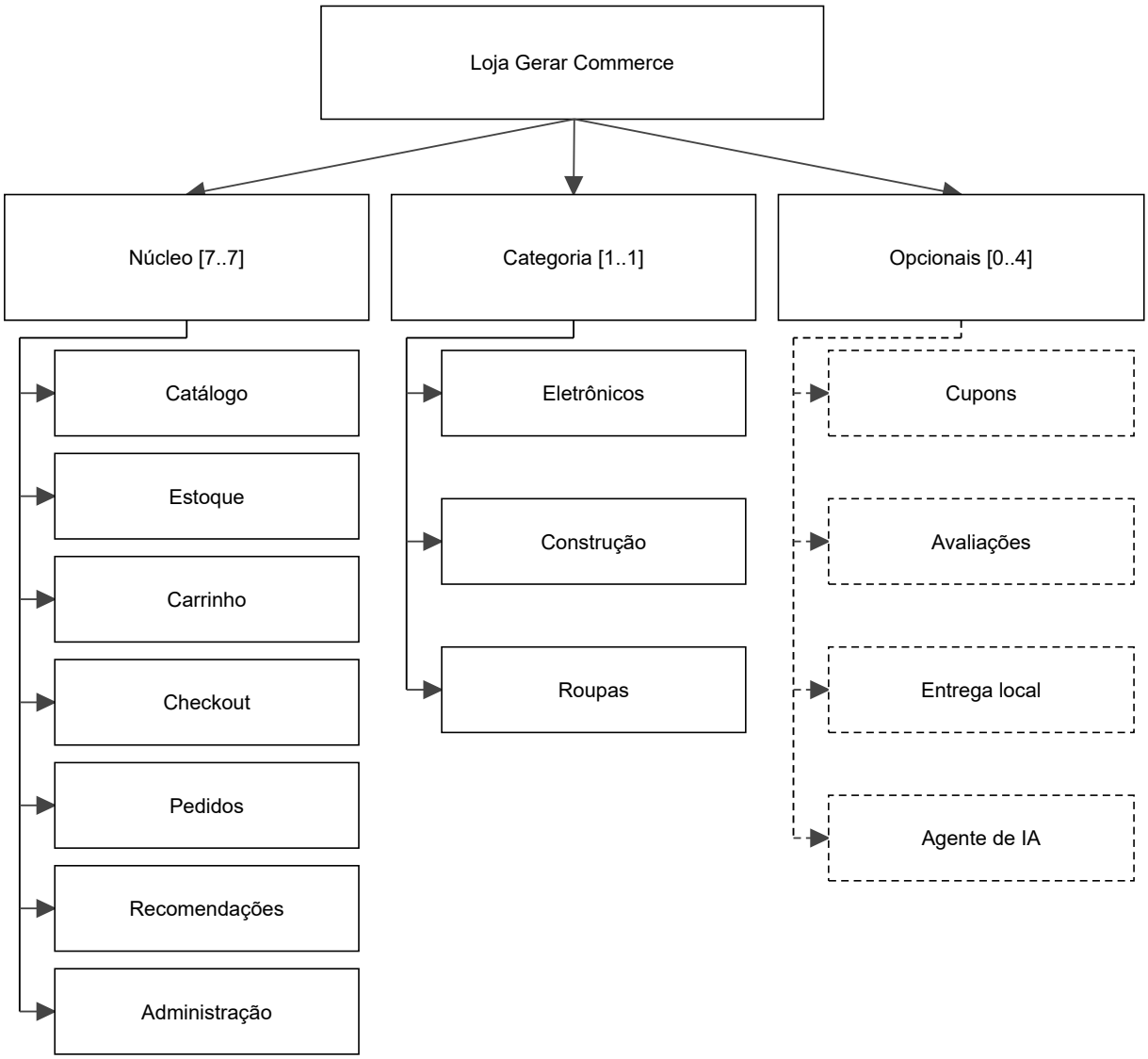
Sprint Backlog I2

A seleção inicial foi PB-008 a PB-014 e PB-017. Persistência, catálogo, reservas, carrinho, checkout, pedidos e administração compunham o fluxo. Em 05/11, houve antecipação dessas atividades e de PB-024 e PB-026. PB-028 a PB-032 registraram composição dinâmica, onboarding, temas e planos, geração e isolamento. O snapshot dessa data documenta a mudança real de escopo.

Sprint Backlog I3

A seleção inicial incluiu PB-015, PB-016, PB-018, PB-019 e PB-021 a PB-023. PB-033 a PB-043 registraram compradores, experiência e correções. A revisão final também incorporou restauração de recomendações e novos diagramas pelas pull requests 47 e 48. Relatórios, revisão de API e ensaio mantêm critérios de aceite próprios. Os registros estão em `docs/gestao/ciclos.md` e `docs/gestao/snapshots/`.

5.1 Diagrama de features



Cardinalidades: todos os 7; exatamente 1 categoria; qualquer subconjunto dos 4 opcionais.

Figura 1 - Modelo de features com grupos obrigatórios, categoria exclusiva e opcionais independentes.
Fonte: elaboração própria a partir do código do projeto.

A raiz representa uma loja da família. Os sete nós do núcleo são obrigatórios. O grupo de categorias exige exatamente uma escolha. Os quatro opcionais são independentes e podem estar ausentes ou presentes. Nome e capa são parâmetros de identidade, e não novas funcionalidades de negócio.

O espaço estrutural contém 3 categorias multiplicadas por 2 elevado a 4 subconjuntos de opcionais, totalizando 48 combinações. Dessas combinações, três pertencem ao plano Essencial, 21 ao Completo e 24 ao Inteligente. Esse cálculo descreve o modelo permitido; não significa que

as 48 combinações foram todas testadas. Diferentes identidades e catálogos ampliam o número de instâncias sem alterar esse espaço de seleção.

5.2 Arquitetura em camadas e variabilidade

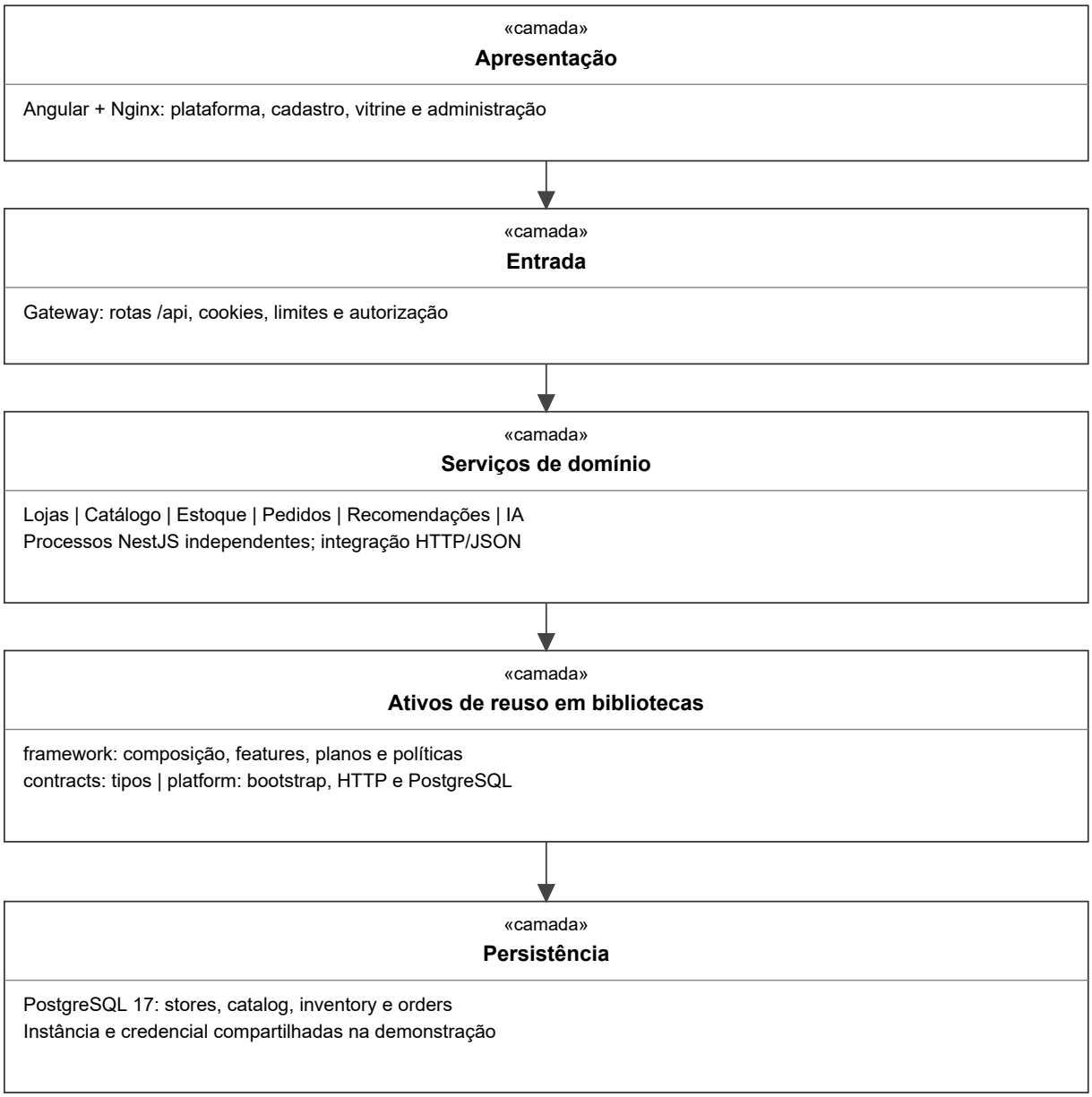


Figura 2 - Camadas da solução e dependências. Os ativos de reuso executam nos serviços consumidores.
Fonte: elaboração própria a partir do código do projeto.

A apresentação usa Angular e Nginx. A entrada centraliza rotas e sessões no gateway. Os serviços de domínio executam regras e orquestram as APIs. Framework, contratos e plataforma são bibliotecas reutilizadas por seus consumidores. PostgreSQL mantém os dados dos quatro domínios persistentes. A camada de reuso participa dos processos consumidores; não é um serviço de rede adicional.

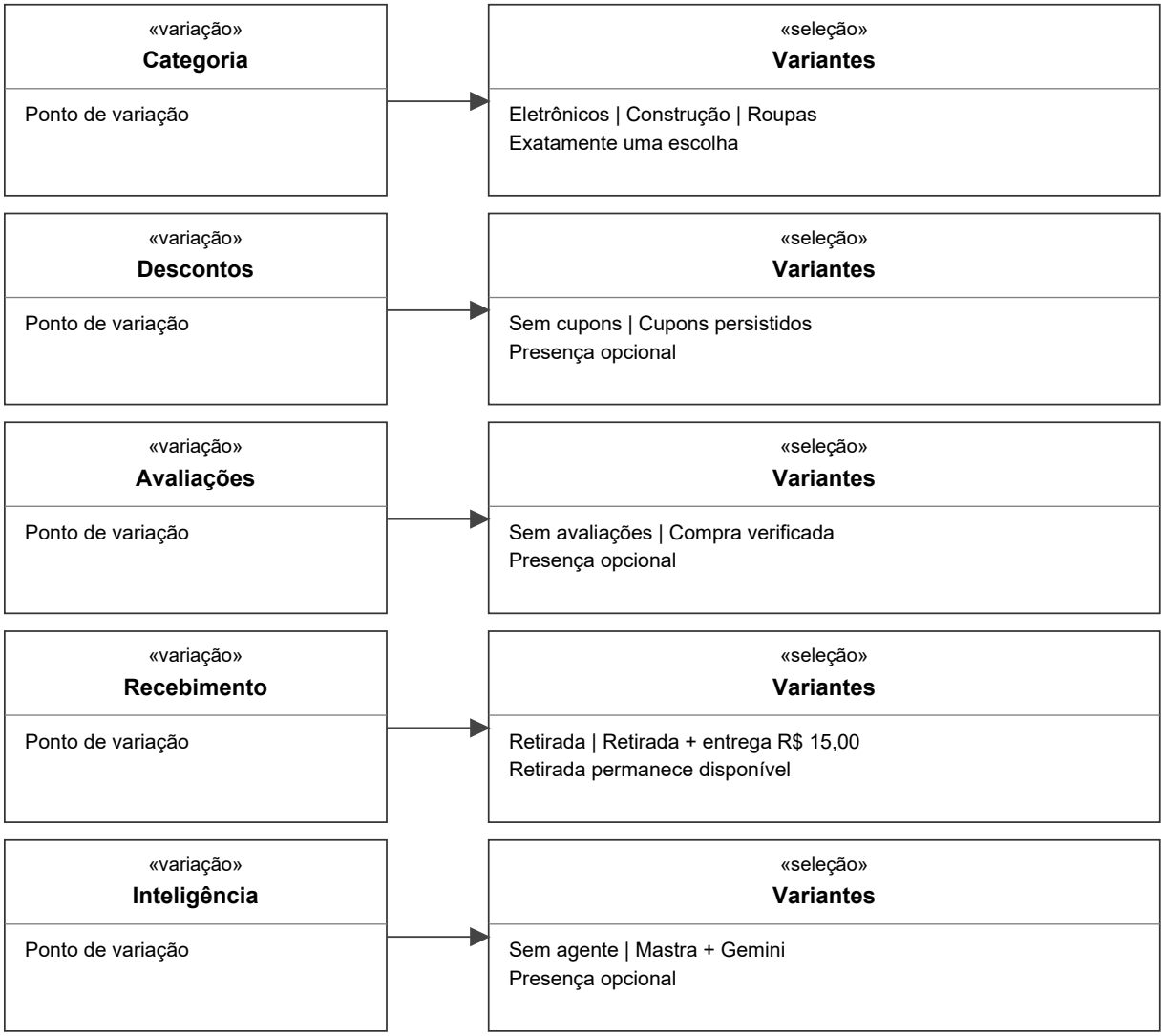


Figura 3 - Pontos de variação, alternativas e restrições. A seleção é vinculada no cadastro e nas configurações.
Fonte: elaboração própria a partir do código do projeto.

Ponto de variação	Variantes e regra	Momento de vínculo
Categoria	Exatamente uma entre Eletrônicos, Construção e Roupas; altera o tema	Cadastro e configurações
Desconto	Sem cupons ou cupons persistidos com PercentageDiscount	Composição da loja e cálculo do checkout
Avaliação	Ausente ou avaliações por compra aprovada	Composição e autorização da publicação
Recebimento	Retirada sempre disponível; delivery acrescenta entrega fixa de R$ 15,00	Composição e escolha da compra

Ponto de variação	Variantes e regra	Momento de vínculo
Inteligência	Sem agente ou chat e agente com Mastra/Gemini	Composição; provedor configurado na implantação
Identidade	Nome, descrição, capa, produtos e slug único	Cadastro; slug permanece fixo

O plano é derivado dos opcionais, não um ponto de seleção independente. O pagamento é fixo e simulado. A restauração de recomendações mantém esse recurso fora da variabilidade opcional: todas as lojas o recebem no blueprint.

5.3 Componentes de software e variabilidade

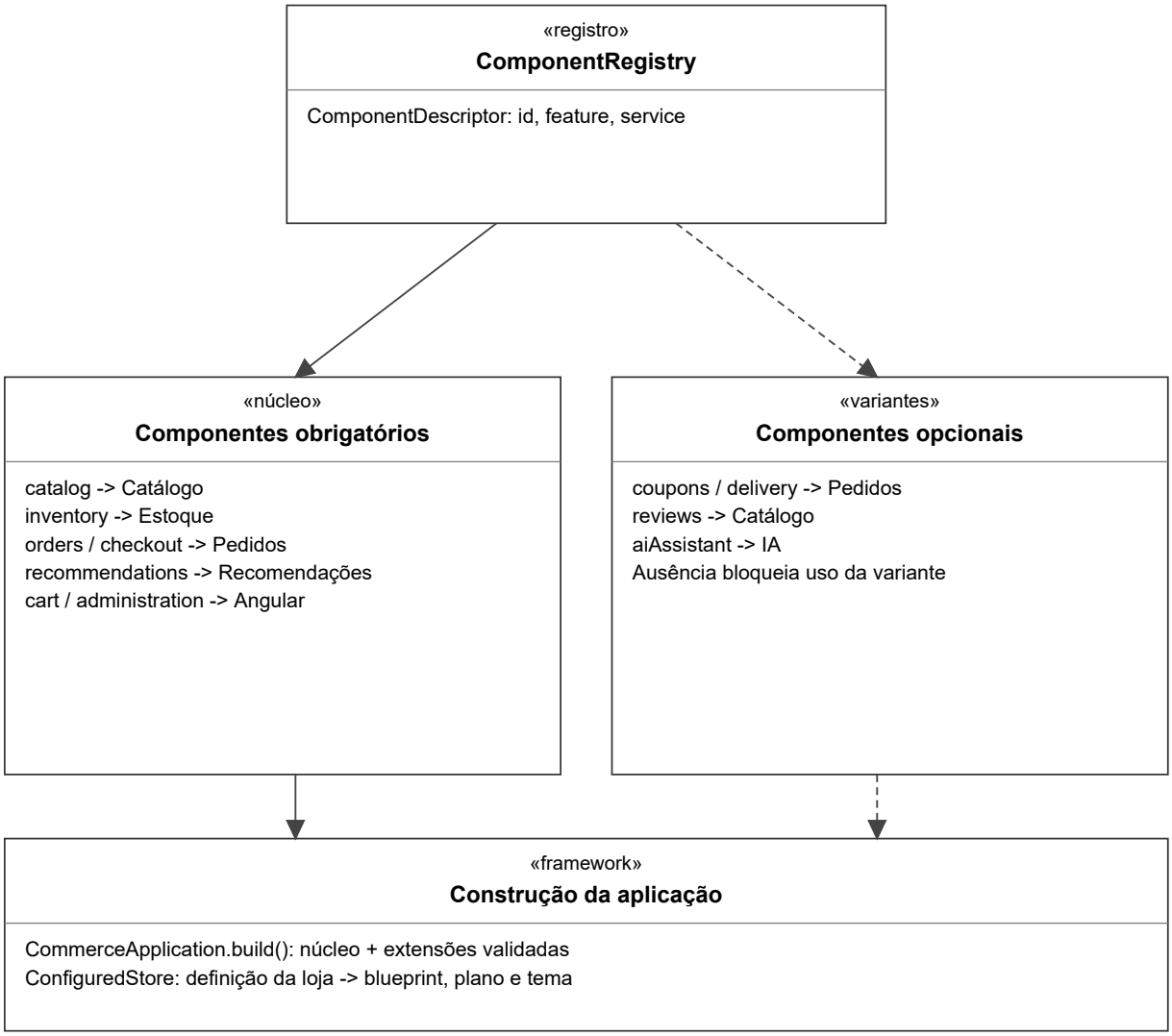


Figura 4 - Componentes e variabilidade da composição. Linhas tracejadas indicam participação condicional.
Fonte: elaboração própria a partir do código do projeto.

O diagrama distingue componentes obrigatórios e opcionais e indica onde seus comportamentos são executados. As linhas tracejadas representam ativação condicional. Mais de uma feature pode pertencer ao mesmo serviço: checkout e pedidos compartilham `orders`; avaliações pertencem a

`catalog`; carrinho e administração ficam no Angular. Logo, contagem de features não corresponde à contagem de microsserviços.

Feature	Responsável registrado	Variação
catalog e inventory	catalog e inventory	Obrigatórios
cart e administration	storefront, a interface Angular	Obrigatórios
orders e checkout	orders	Obrigatórios
recommendations	recommendations	Obrigatório
coupons e delivery	orders	Opcionais independentes
reviews	catalog	Opcional
aiAssistant	ai-assistant	Opcional

`build()` valida a presença do núcleo e a correspondência entre features e componentes. Um descritor contém ID, feature e responsável. Os opcionais são registrados por `ConfiguredStore.registerExtensions()`. A interface adapta os controles e os serviços aplicam as guardas; somente ocultar um botão não autoriza ou bloqueia uma operação.

O pacote é compartilhado por npm workspaces. Na instalação, `@gerar/framework` é vinculado ao diretório local. O build compila bibliotecas antes dos serviços e a API pública exporta composição e políticas. Assim, as aplicações reutilizam os mesmos ativos, preservando processos de execução e dados de configuração próprios de cada contexto.

5.4 Tela de configuração e construção de um produto

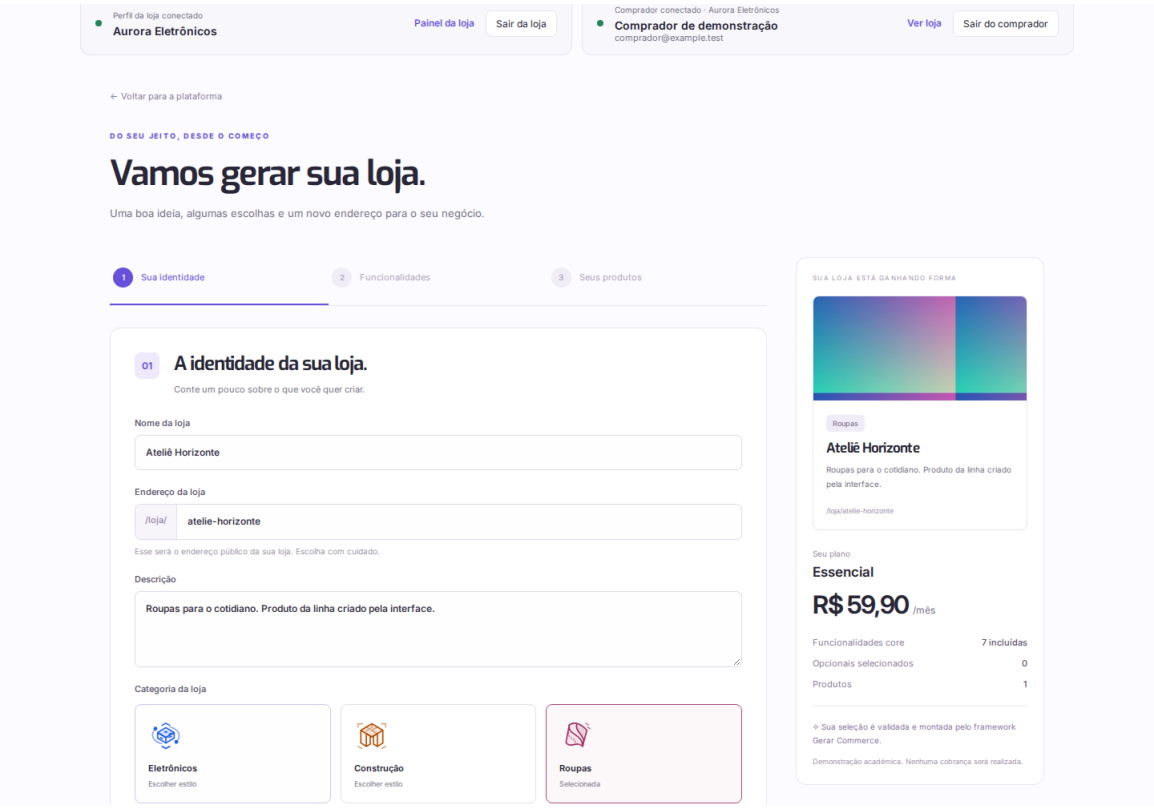


Figura 5 - Identidade do novo produto Ateliê Horizonte, com categoria Roupas e capa fictícia.
Fonte: captura da aplicação com dados fictícios.

O cadastro em `/criar` reúne nome, descrição, slug, capa e senha. A categoria escolhe a apresentação visual. A senha administrativa não se confunde com a conta de comprador. O requisito de pelo menos um produto evita publicar uma loja vazia.

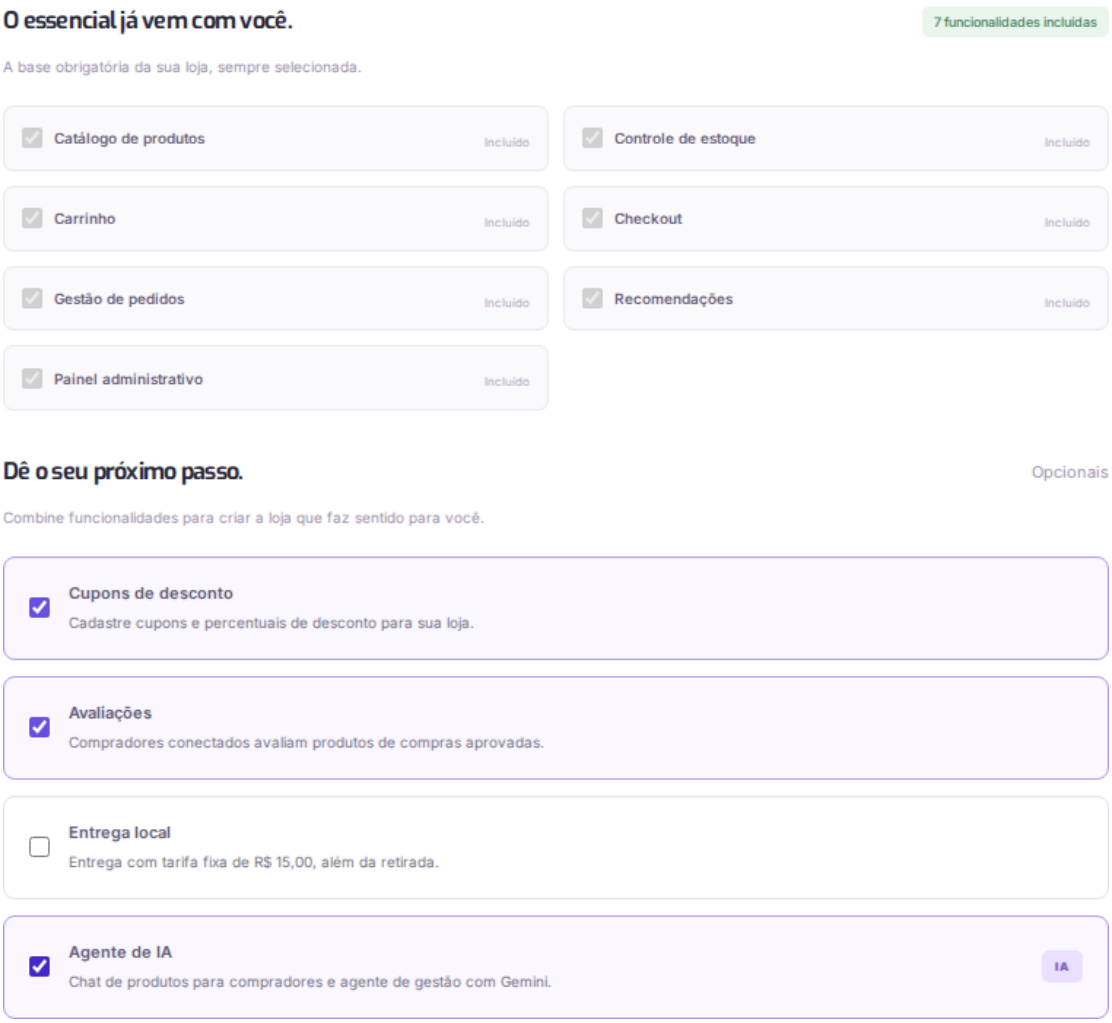


Figura 6 - Seletor real da revisão com sete features obrigatórias. Cupons, avaliações e IA foram selecionados; entrega permaneceu desmarcada.
Fonte: captura da aplicação com dados fictícios.

O núcleo aparece marcado e bloqueado para remoção. A seleção usada na demonstração gera dez features: sete obrigatórias e três opcionais. Com IA, o plano calculado é Inteligente, por R$ 99,90. A entrega continua sendo retirada, porque `delivery` não foi selecionada.

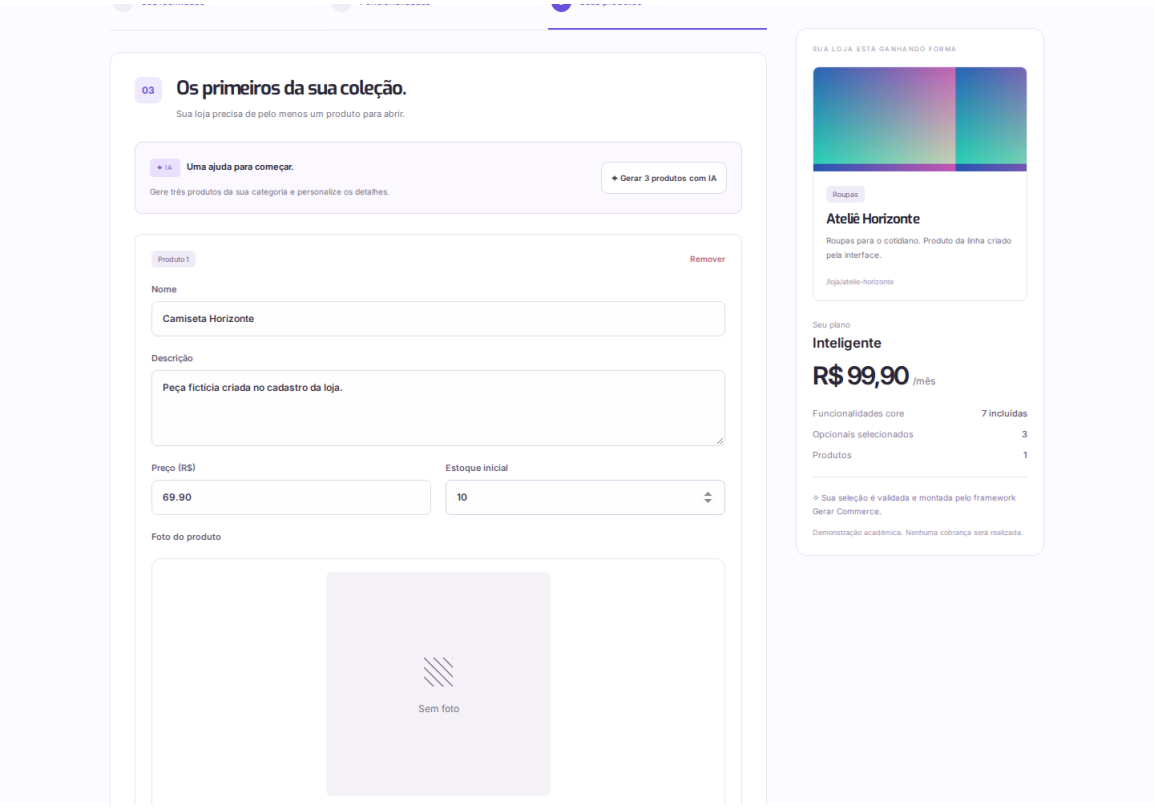


Figura 7 - Inclusão do produto Camiseta Horizonte com preço de R$ 69,90 e dez unidades, antes da publicação.
Fonte: captura da aplicação com dados fictícios.

Na etapa final, o lojista informa os produtos ou solicita três rascunhos por IA. A ajuda inicial está disponível no cadastro e não obriga contratar o agente da loja. Sem configuração Gemini, os rascunhos são exemplos explicitamente identificados. Os dados são editáveis antes de enviar o cadastro.

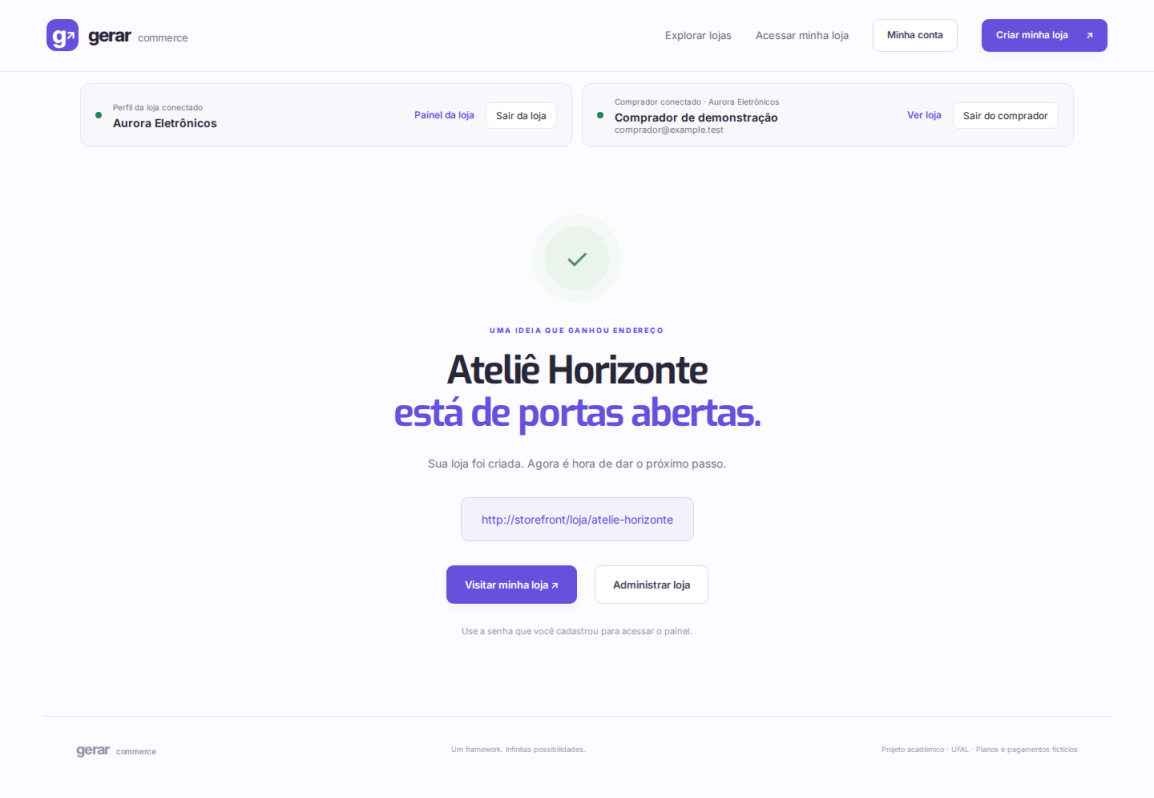


Figura 8 - Confirmação real de publicação. O produto da linha passa a ter vitrine e administração por slug.
Fonte: captura da aplicação com dados fictícios.

O serviço de lojas usa `composeStore()`, grava o cadastro inativo, provisiona catálogo e estoque e então o ativa. Em falha, tenta compensar o provisionamento. A figura comprova o resultado pela interface; a leitura da API confirmou slug, opções e plano do produto criado.

```
// Definição equivalente à composição utilizada no cadastro validado.
const produto = composeStore({
  slug: 'atelie-horizonte', name: 'Ateliê Horizonte',
  category: 'roupas',
  options: ['coupons', 'reviews', 'aiAssistant'],
});
// 7 obrigatórias + 3 opcionais; plano de 9990 centavos.
// A publicação efetiva também persiste identidade e produtos.
```

6 Arquitetura baseada em microsserviços

6.1 Arquitetura e recomendações por histórico

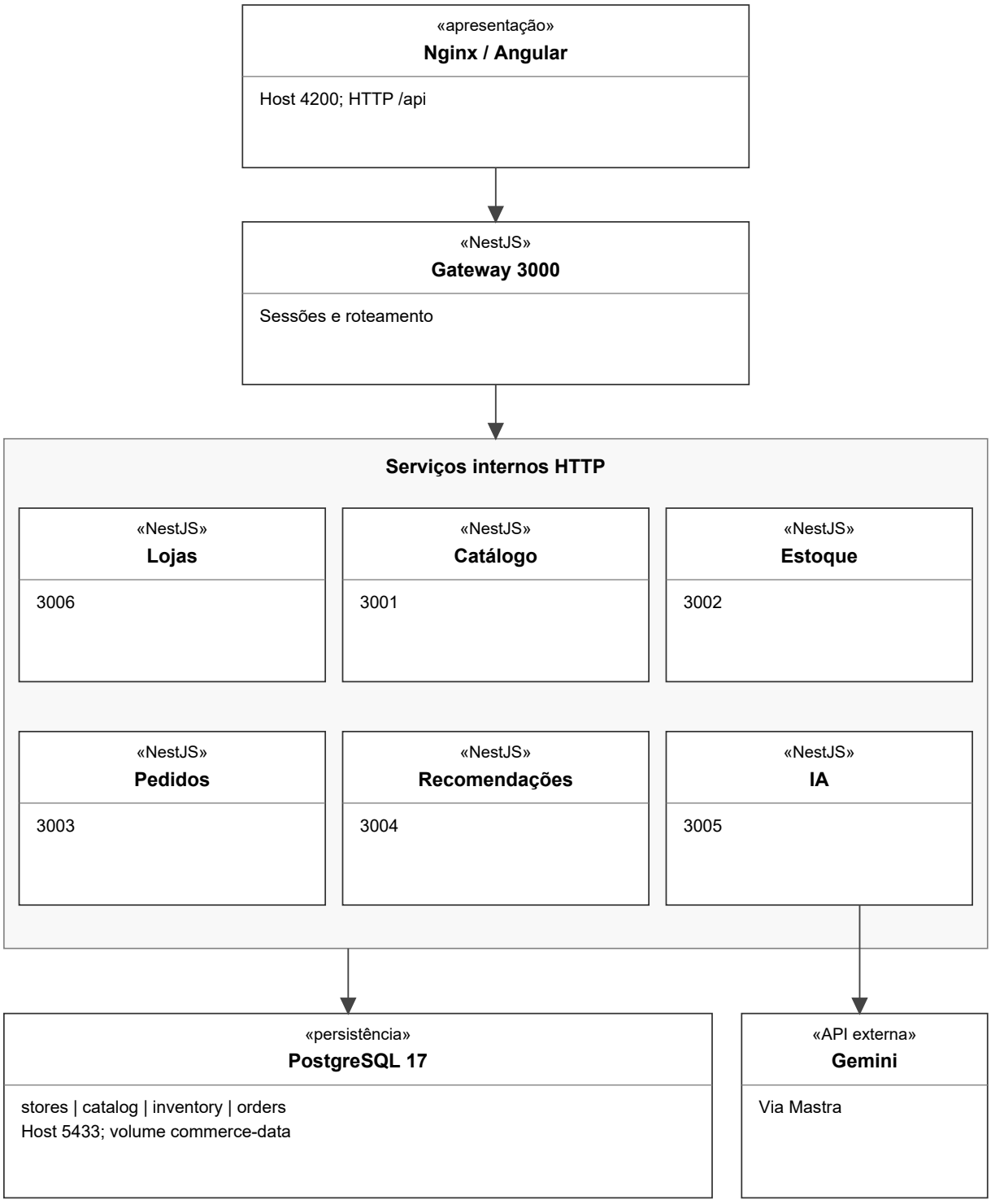


Figura 9 - Arquitetura com sete processos NestJS, apresentação e banco, totalizando nove contêineres.
Fonte: elaboração própria a partir do código do projeto.

A implantação contém nove contêineres: Angular servido por Nginx, gateway, seis serviços internos e PostgreSQL. Os sete processos NestJS usam as portas 3000 a 3006. O navegador usa a

API do gateway pela mesma origem. As chamadas entre domínios usam HTTP/JSON, e os processos compartilham infraestrutura por bibliotecas.

PostgreSQL 17 usa schemas `stores`, `catalog`, `inventory` e `orders`. Cada domínio consulta suas próprias tabelas. Na demonstração há uma única instância e uma credencial compartilhada; o isolamento por schema organiza responsabilidades, mas não equivale a credenciais de banco distintas por serviço [7]. Recomendações, gateway e IA não mantêm tabelas próprias de domínio.

Recomendação independente de IA

`RecommendationsController` atende na porta 3004. Para sugerir produtos, consulta catálogo, saldos, histórico aprovado do comprador e cestas aprovadas da loja. Primeiro remove itens sem estoque e produtos já comprados pela conta; depois calcula a pontuação e retorna até três candidatos com justificativas. O ranking funciona sem chave Gemini.

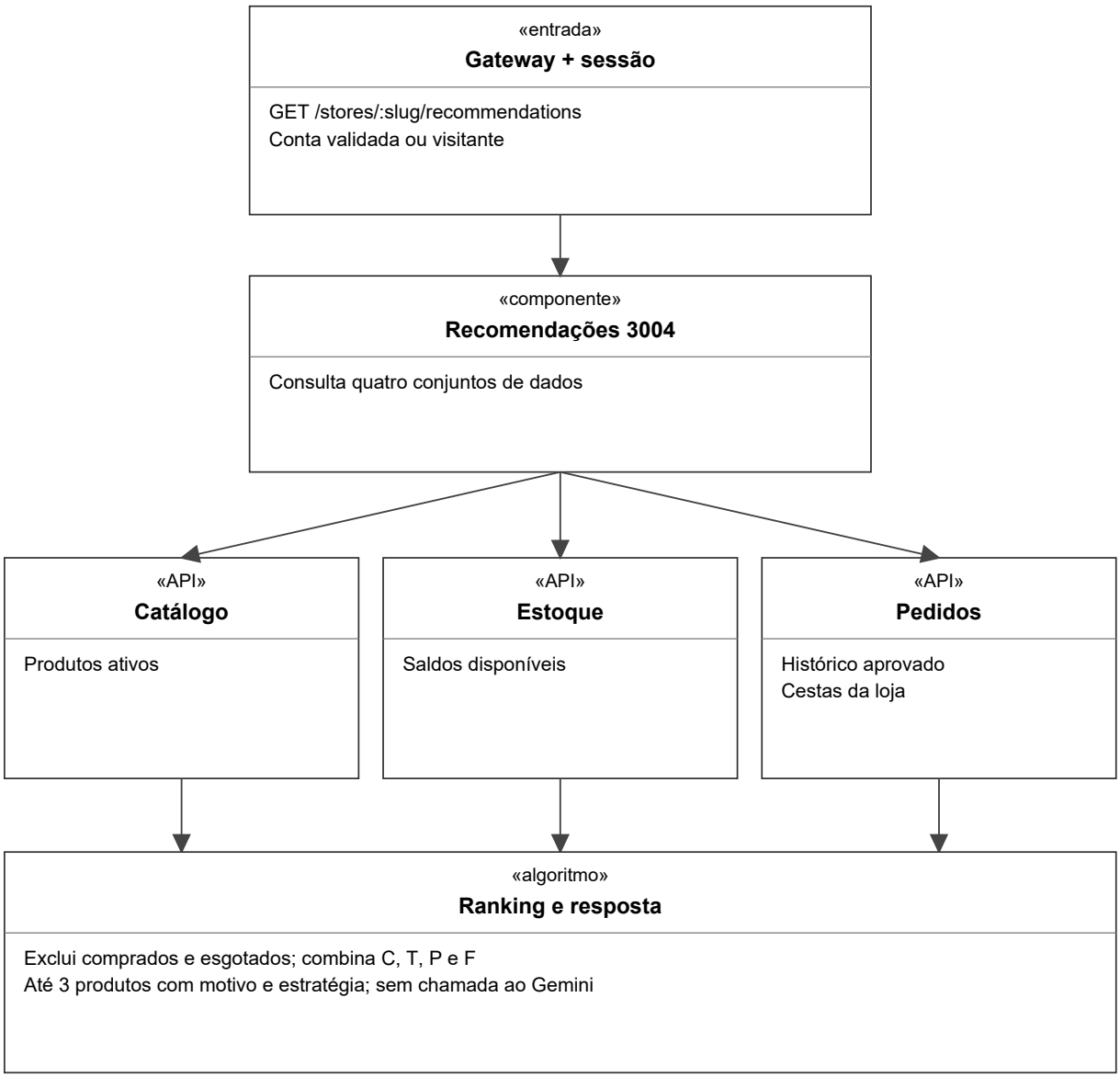


Figura 10 - Fluxo da recomendação: identidade por sessão, APIs de domínio, filtragem e ranking explicável.
Fonte: elaboração própria a partir do código do projeto.

A rota pública normal é `GET /api/stores/:slug/recommendations`. O gateway obtém a conta pelo cookie, quando válido; sem sessão utiliza o perfil de visitante. A rota que inclui um ID de comprador exige autenticação e rejeita ID diferente da conta. Pedidos fornece apenas compras aprovadas da própria loja. As cestas retornam IDs dos produtos, sem os dados de identificação dos compradores.

Sinais e cálculo da pontuação

Símbolo	Sinal implementado	Interpretação
C	Fração das linhas históricas na categoria do candidato	Afinidade com categorias compradas
T	Cestas que contêm o candidato e algum item já comprado	Evidência de compra conjunta na loja
P	Cestas aprovadas que contêm o candidato	Popularidade por presença em pedidos
N	Número de cestas consultadas, com denominador mínimo um	Normalização dos sinais coletivos
F	1 menos a distância relativa ao preço médio, limitada ao intervalo de zero a um	Proximidade de preço

O cálculo corresponde ao seguinte trecho central do serviço:

Fonte: `backend/services/recommendations/src/main.ts`.

```
const score =
  weights.category * category +
  weights.coPurchase * (together.length / Math.max(1, baskets.length)) * 10 +
  weights.popularity * (popularity / Math.max(1, baskets.length)) * 10 +
  weights.price * priceFit;
```

Em termos dos símbolos da tabela, S = 3C + 20T/N + 5P/N + 0,5F. Os fatores dez do código fazem parte da escala dos sinais de cestas. O histórico considera até 100 pedidos aprovados da conta, e as cestas usam até 300 pedidos aprovados da loja. As frequências são por linha ou presença em cesta, sem multiplicar pela quantidade de unidades. “Mais comprado”, na interface, corresponde a essa frequência em pedidos.

A justificativa usa prioridade própria: compra conjunta quando há cestas relacionadas; afinidade de categoria quando positiva; popularidade quando existente; caso contrário, escolha inicial do catálogo. A estratégia global retorna `history-category` quando alguma sugestão usa sinal pessoal, `store-popularity` quando há cestas sem sinal pessoal ou `catalog-fallback` quando não há cestas. O nome da estratégia pessoal também abrange compra conjunta, apesar de mencionar categoria.

Exemplo verificado na demonstração

A conta principal comprou Sensor Alpha, por R$ 100,00. Outra conta comprou Sensor Alpha com Hub Beta e depois Fone Sigma. Uma tentativa com Cabo Delta foi recusada. O catálogo também tinha um produto esgotado. As três compras aprovadas produziram três cestas para o cálculo.

Candidato para a conta principal	C	T	P	F	Pontuação aproximada
Hub Beta, R$ 120,00	1	1	1	0,8	11,73
Fone Sigma, R$ 80,00	1	0	1	0,8	5,07
Cabo Delta, R$ 30,00	1	0	0	0,3	3,15

Os valores da tabela são calculados a partir da massa e da fórmula; a API não expõe pontuação. A resposta validada colocou Hub Beta em primeiro lugar, retirou Sensor Alpha por já ter sido comprado e excluiu o item esgotado. A compra recusada não compôs as cestas aprovadas. Como todos os produtos da loja receberam a mesma categoria, o sinal C é igual nesse exemplo; compra conjunta, popularidade e preço discriminam a ordem.

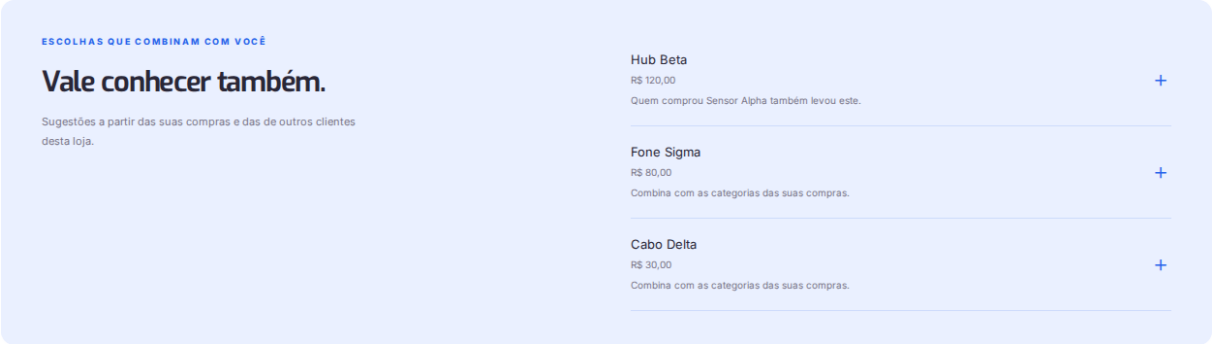


Figura 11 - Recomendação personalizada real após compras aprovadas. O motivo de Hub Beta relaciona a sugestão a Sensor Alpha.
Fonte: captura da aplicação com dados fictícios.

Para visitante, não há histórico pessoal; popularidade ordenou Sensor Alpha em primeiro. Quando não existem compras aprovadas, o serviço oferece candidatos do catálogo em estoque. Empates não possuem um critério secundário explícito no ranking e preservam a ordem recebida do catálogo.



Figura 12 - Recomendações para visitante sem sessão: a interface apresenta os itens mais presentes nas cestas aprovadas da loja.
Fonte: captura da aplicação com dados fictícios.

O algoritmo é uma heurística de ranking, não um modelo treinado nem uma prova de preferência real. O conjunto consultado é limitado e uma loja com poucos produtos pode devolver menos de três sugestões. O chat opcional de Gemini recebe contexto e conversa com o comprador; ele não substitui esse microsserviço obrigatório.

6.2 Especificação documentada da API gateway

Todas as rotas têm prefixo `/api` e retornam JSON. Na tabela, L abrevia `/stores/:slug`. Sessões administrativas e de compradores possuem cookies diferentes, delimitados pela loja. O gateway valida autorização, encaminha as operações e preserva os erros dos serviços.

Método e rota pública	Contrato e regra de acesso
GET /features e /stores	Features, categorias e lojas ativas; público.
POST /stores	Identidade, capa, senha, categoria, opções e 1 a 30 produtos; limite por IP.
POST /generate-products	Nome/categoria; três rascunhos com origem explícita.
GET L	Identidade e blueprint; sem senha ou sessão.
GET L/products e L/products/:id	Catálogo ou detalhe da própria loja; público.
GET L/stock	Lista productId e available; público.
GET L/recommendations	Conta identificada pela sessão ou visitante; até três sugestões.
GET L/recommendations/:customerId	Exige comprador e ID correspondente à própria conta.
POST L/login e L/logout; GET L/session	Senha e sessão administrativa; cookie específico da loja.
PUT L/settings	Identidade, capa, categoria e opções; exige administração; slug fixo.
POST L/products; PUT/DELETE L/products/:id	CRUD de produto e saldo; exige administração.
GET L/orders	Até 100 pedidos; exige administração.

Método e rota pública	Contrato e regra de acesso
POST L/customers e L/customer-login	Conta com nome, e-mail e senha; login do comprador.
GET L/customer-session e L/customer-orders	Perfil e até 100 pedidos pessoais; comprador autenticado.
POST L/customer-logout	Revoga somente a sessão do comprador.
POST L/checkout	Referência, itens, pagamento e recebimento; comprador autenticado.
GET L/coupons; PUT/DELETE L/coupons/:code	Códigos persistidos; administração e feature coupons.
POST L/coupon-quote	Código e subtotal para prévia; público com coupons.
GET/POST L/reviews/:id	Leitura pública; escrita com login e compra aprovada; exige reviews.
GET L/review-eligibility/:id	Consulta compra aprovada; comprador e reviews.
POST L/chat	Mensagem/histórico curto; texto e links; público com aiAssistant.
GET L/assistant-status; POST L/assistant	Estado sem segredos; pergunta ao agente; administração e IA.
GET /health e /services	Saúde do gateway e dos cinco serviços internos.

O trecho abaixo documenta a entrada normal das recomendações. A identidade é obtida no servidor; um visitante não escolhe livremente o histórico de outra pessoa.

Fonte: `backend/services/gateway/src/main.ts`.

```
@Get('stores/:slug/recommendations') async suggestions(
  @Param('slug') slug: string,
  @Req() req: any,
) {
  await store(slug);
  let id = 'visitante';
  if (token(req, slug, true)) {
    try {
      id = (await this.customerSession(slug, req)).id;
    } catch (error) {
      if (!(error instanceof HttpException) || error.getStatus() !== 401) throw error;
    }
  }
  return callService('recommendations', path(slug, '/recommendations/' + id));
}
```

IDs de loja e produtos, quantidades e textos passam por validação. Valores monetários são centavos inteiros. Erros relevantes incluem 400 por dados inválidos, 401 por sessão ausente, 403 por autorização, 404 por recurso inexistente, 409 por conflito, 429 por limite e 502/503 por dependência indisponível. Swagger está em `/docs` de cada processo, com detalhamento de DTOs ainda parcial.

6.3 Especificação de cada microsserviço

Na tabela, S abrevia `/internal/stores/:slug`. As rotas internas usam chave entre serviços. Os serviços não são publicados diretamente no host pelo Compose. Bootstrap, cliente HTTP, validação e acesso ao PostgreSQL pertencem a `platform`, mantendo regras de negócio nos domínios.

Componente e rotas	Responsabilidade e comportamento
Lojas: GET /features, /stores, /stores/:slug; POST /internal/stores; PUT S	Seleção, cadastro e atualização; ativa após provisionar produtos e saldos.
Lojas: POST S/login e S/session; DELETE S/session	Senha scrypt; token aleatório com hash e validade.
Compradores: POST S/customers, S/customer-login e S/customer-session; DELETE S/customer-session	Conta por loja e e-mail normalizado; sessão independente.
Catálogo: GET /stores/:slug/products e /products/:id; POST S/seed e S/products; PUT/DELETE S/products/:id; DELETE S	Consulta, CRUD e compensação; retirada lógica preserva histórico.
Avaliações: GET /stores/:slug/reviews/:id; POST S/review-eligibility/:id e S/reviews/:id	Sessão validada por lojas e compra aprovada consultada em pedidos.
Estoque: GET /stores/:slug/stock; POST S/seed; PUT S/stock/:id; DELETE S	Consulta, carga inicial, ajuste e compensação.
Reservas: POST S/reserve e S/reservations/:reference/:action	Reserva atômica; action confirm ou release; idempotência por referência.
Pedidos: POST S/checkout; GET S/orders, S/customer-orders/:customerId e S/purchases/:customerId/:productId	Cotação oficial, tentativa durável, histórico e comprovação de compra.
Cupons: GET S/coupons; PUT/DELETE S/coupons/:code; POST S/coupon-quote	Percentual de 1 a 100, compra mínima, ativação e validade.
Recomendações: GET /stores/:slug/recommendations/:customerId	Consulta catálogo, estoque e pedidos; filtra e pontua até três candidatos.
Pedidos: GET S/history/:customerId e S/baskets	Até 100 pedidos aprovados da conta e 300 cestas aprovadas da loja.
IA: GET /internal/status; POST /internal/generate-products	Estado da configuração e três rascunhos.
IA: POST S/shopping-chat e S/query	Chat limitado ao catálogo; agente administrativo com ferramentas.
Todos: GET /api/health; Swagger /docs	Nome, porta, armazenamento declarado e documentação do processo.

Lojas controla cadastro inativo, provisionamento, ativação e composição. Catálogo mantém produtos e avaliações por compra aprovada. Estoque usa transações e atualização condicional para reservas, com confirmação e liberação idempotentes. Pedidos persiste a intenção antes de reservar e usa referência para retentativas. A mesma referência concluída devolve o mesmo pedido; alterar o conteúdo exige outra referência.

Recomendações deriva os dados pelas APIs e não grava um perfil próprio. IA usa `Agent` e `createTool` do Mastra para perguntas administrativas. Suas ferramentas de estoque e vendas capturam a loja no contexto do servidor e são de leitura [8]. O consultor público recebe catálogo e saldos sem ferramentas administrativas. Não há memória persistente de Mastra nessa entrega.

Os trechos do relatório são excertos explicativos. A seção 9 oferece links para o código completo do gateway, dos sete serviços e das bibliotecas, evitando reproduzir listagens integrais.

6.4 Serviços em execução e respectivas portas

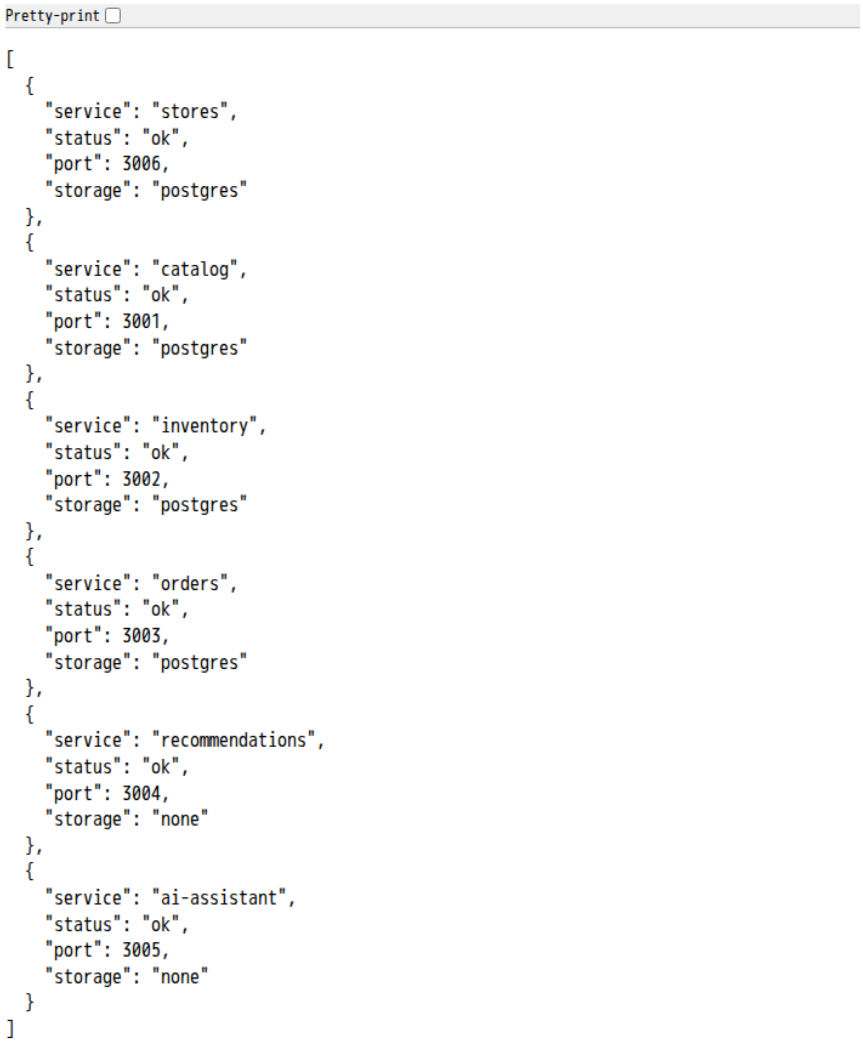


Figura 13 - Consulta real ao agregador /api/services na revisão validada. Os seis serviços internos responderam com suas portas e status ok.
Fonte: captura da aplicação com dados fictícios.

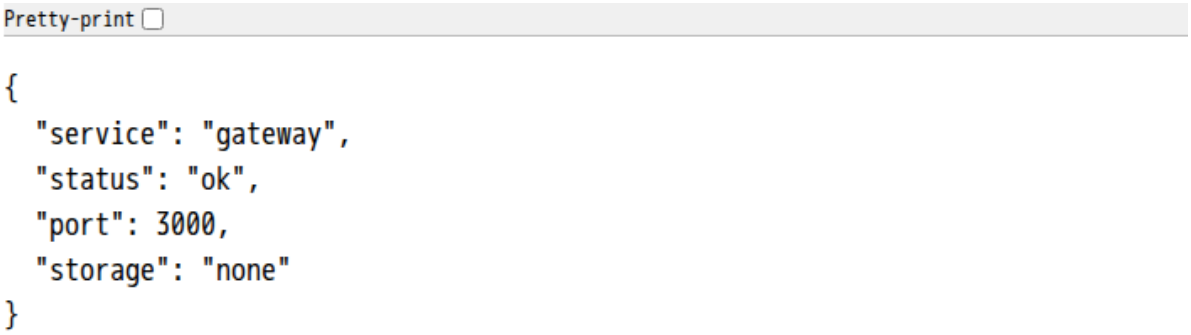


Figura 14 - Resposta real de /api/health do gateway, completando os sete processos NestJS da arquitetura.
Fonte: captura da aplicação com dados fictícios.

As respostas foram apenas formatadas com indentação no navegador para leitura. A camada de apresentação atende na porta 4200 do host e a API, na 3000. No ambiente isolado dos testes, a rede é interna e não publica portas no host.

Processo	Porta interna	Publicação padrão no host
Angular em Nginx	80	4200
Gateway	3000	3000
Catálogo	3001	Não publicada
Estoque	3002	Não publicada
Pedidos	3003	Não publicada
Recomendações	3004	Não publicada
IA	3005	Não publicada
Lojas e contas	3006	Não publicada
PostgreSQL	5432	5433, limitado a 127.0.0.1

6.5 Passo a passo para construir outra aplicação

Primeiro, o lojista escolhe uma categoria e um subconjunto de opcionais. Para Casa Nova Construção, seleciona Construção, cupons e entrega. O núcleo é mantido e o framework determina plano Completo e política de entrega fixa. A composição isolada pode ser inspecionada em código:

```
import { composeStore, quoteCheckout } from '@gerar/framework';
const loja = composeStore({
  slug: 'casa-nova', name: 'Casa Nova Construção',
  category: 'construcao', options: ['coupons', 'delivery'],
});
// Sete features obrigatórias e dois opcionais.
// Plano: 7990 centavos. Tarifa de entrega: 1500 centavos.
```

Segundo, a publicação ocorre pelo formulário `/criar` ou pelo contrato de cadastro. O envio abaixo é ilustrativo; `capaPng` deve conter uma imagem PNG/JPG/WebP válida em data URL, ou pode ser substituído por URL HTTPS direta. O endereço precisa estar livre, e a senha é apenas um exemplo acadêmico.

```
// POST /api/stores, pela origem da plataforma.
const cadastro = {
  slug: 'casa-nova', name: 'Casa Nova Construção',
  description: 'Materiais para pequenos projetos.',
  category: 'construcao', options: ['coupons', 'delivery'],
  cover: capaPng, password: 'exemplo-academico-2026',
  products: [{
    name: 'Kit de ferramentas',
    description: 'Produto fictício para demonstração.',
    priceInCents: 12990, stock: 12, image: '',
  }],
};
const response = await fetch('/api/stores', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(cadastro),
});
if (!response.ok) throw new Error('Cadastro não concluído.');
```

Terceiro, o serviço valida, compõe e provisiona catálogo e estoque. Após a ativação, a vitrine fica em `/loja/casa-nova` e o painel em `/admin/casa-nova`. O lojista entra, cadastra um cupom e ajusta produtos e saldo. Para comprar, cria-se uma conta de comprador da loja; o checkout consulta preços oficiais e aplica as políticas.

Quarto, o produto pode evoluir pelas configurações: alterar categoria ou opcionais recompõe a definição e recalcula o plano. O slug permanece fixo. Não é necessário gerar outro frontend, copiar serviços ou executar um build por lojista. `npm run derive -- casa-nova`, no backend, exporta um snapshot de uma loja existente; não é o mecanismo de publicação.

Para uma categoria inédita, o desenvolvedor amplia temas e apresentação. Para uma nova feature, precisa ampliar contrato, catálogo, validação, registro, implementação de domínio, guardas, interface e testes. Um novo descritor sozinho não implementa comportamento. Essa distinção separa a configuração de um produto da evolução dos ativos da linha.

7 Tecnologias da implementação

Angular 22.2.4, com CLI/build 22.2.6, compõe a interface; NestJS 12.3.3 organiza os serviços TypeScript [9, 10]. O backend exige Node.js a partir de 24.15 e anterior a 25. O framework usa TypeScript 6.0.3 e módulos ECMAScript. As versões se referem aos manifests da revisão documentada.

Microsserviço	Linguagem e bibliotecas	Banco de dados
gateway	TypeScript, Node 24, NestJS	Sem persistência própria; limites em memória
stores	TypeScript, NestJS e crypto scrypt	PostgreSQL 17, schema stores

Microsserviço	Linguagem e bibliotecas	Banco de dados
catalog	TypeScript e NestJS	PostgreSQL 17, schema catalog
inventory	TypeScript e NestJS	PostgreSQL 17, schema inventory
orders	TypeScript e NestJS	PostgreSQL 17, schema orders
recommendations	TypeScript e NestJS; ranking determinístico para as mesmas entradas	Sem banco próprio; consulta APIs
ai-assistant	TypeScript, NestJS, Mastra 1.82.0 e Zod 4.6.7	Sem banco próprio; Gemini e APIs

O driver `pg` oferece pool e transações. Migrações SQL idempotentes criam e adaptam tabelas durante a inicialização. JSONB armazena documentos de lojas, produtos e pedidos junto de chaves e restrições relacionais. Senhas e tokens são persistidos como hashes, e os perfis de comprador e administrador mantêm sessões separadas.

Docker Compose descreve serviços, rede, dependências de saúde e volume [11]. Nginx serve o build Angular e encaminha `/api` ao gateway. O volume `commerce-data` preserva cadastros ao reiniciar ou recriar contêineres. Sua remoção explícita apaga o banco. A sacola é local ao navegador e usa `localStorage` por loja.

8 Funcionamento dos produtos por telas

8.1 Plataforma e escolha de produto

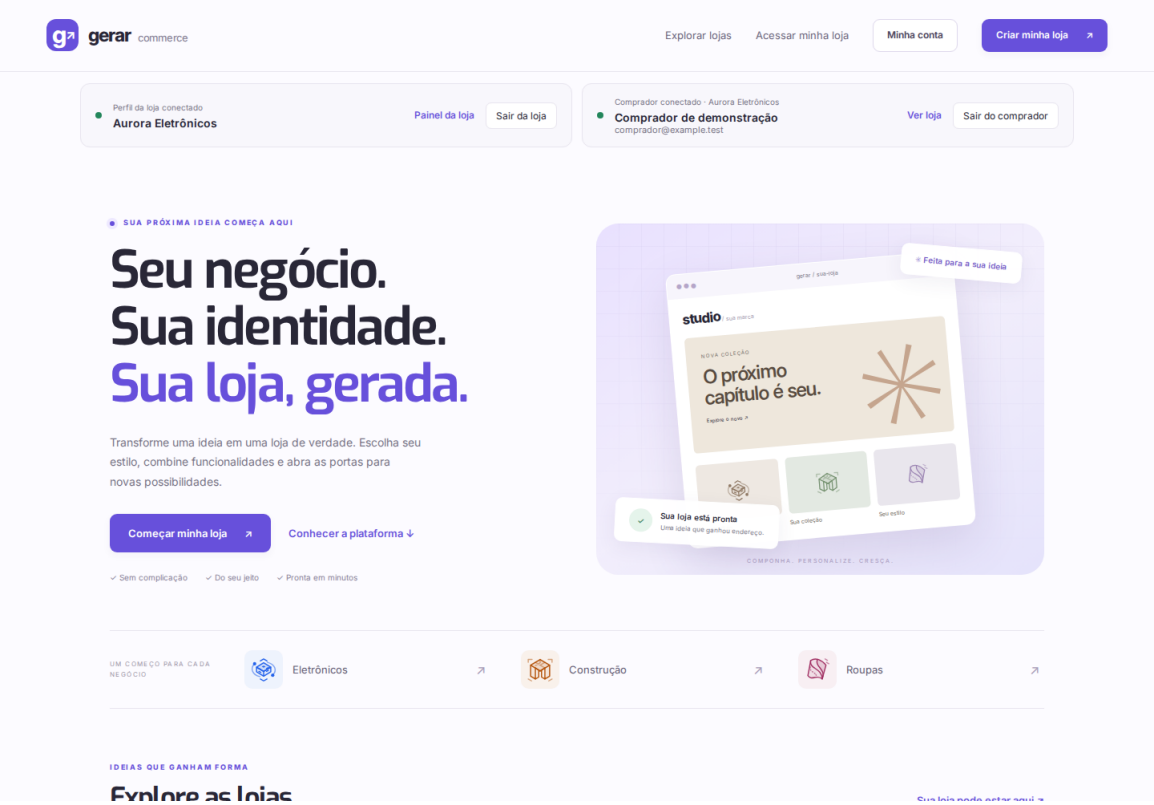


Figura 15 - Plataforma com lojas fictícias de categorias distintas. Apresentação Angular consulta gateway e stores.
Fonte: captura da aplicação com dados fictícios.

A página principal apresenta as três categorias e as lojas ativas. O usuário pode buscar uma loja ou iniciar o cadastro. O mesmo frontend atende toda a família. As telas da seção 5.4 mostram a seleção e a construção efetiva do novo produto.

8.2 Loja essencial e recomendações

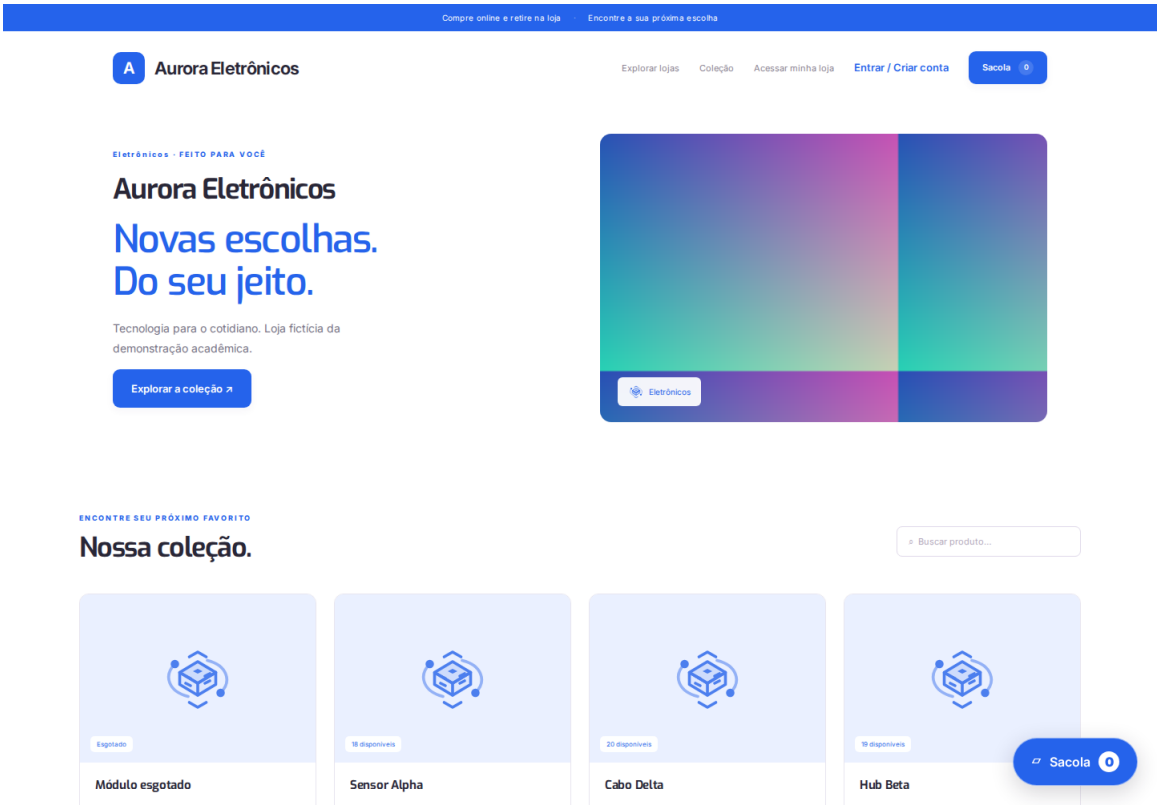


Figura 16 - Aurora Eletrônicos, produto Essencial com tema azul. A vitrine reúne catálogo, estoque e recomendações pelo gateway.
Fonte: captura da aplicação com dados fictícios.

A loja essencial oferece os sete recursos obrigatórios. O comprador consulta produtos e saldos, abre detalhes e usa a sacola. A seção “Vale conhecer também” consulta `recommendations`, que combina APIs de catálogo, estoque e pedidos. Ao entrar com a conta, a resposta pode usar suas compras aprovadas. As figuras de recomendação da seção 6.1 mostram as duas situações efetivamente verificadas.

8.3 Administração e fluxo de compra

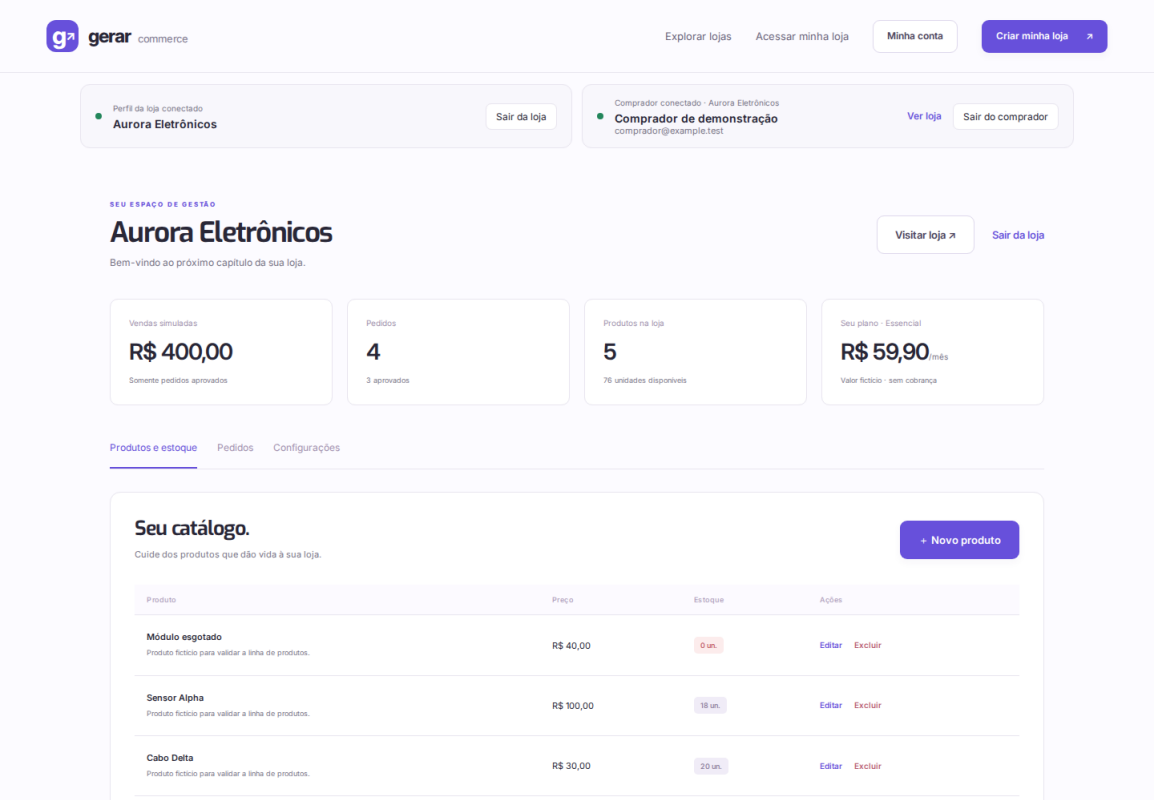


Figura 17 - Painel da Aurora Eletrônicos, com três pedidos aprovados, uma recusa e dados fictícios. O gateway agrega stores, catalog, inventory e orders.
Fonte: captura da aplicação com dados fictícios.

O painel exige a senha da loja. Produtos, saldos, pedidos e configurações usam os mesmos componentes em todas as variantes. Na compra, a conta de comprador é validada em lojas; pedidos consulta catálogo, usa o framework para cotar e solicita a reserva ao estoque. Aprovação confirma e recusa libera. A conta administrativa não autoriza uma compra em nome de um comprador.

Cupons e entrega ficam disponíveis somente quando selecionados. Avaliações exigem `reviews`, conta autenticada e compra aprovada do produto; o nome do autor vem da conta. Uma pessoa mantém uma avaliação por produto, atualizável. Sessões e dados são delimitados pela loja.

8.4 Produto com inteligência artificial

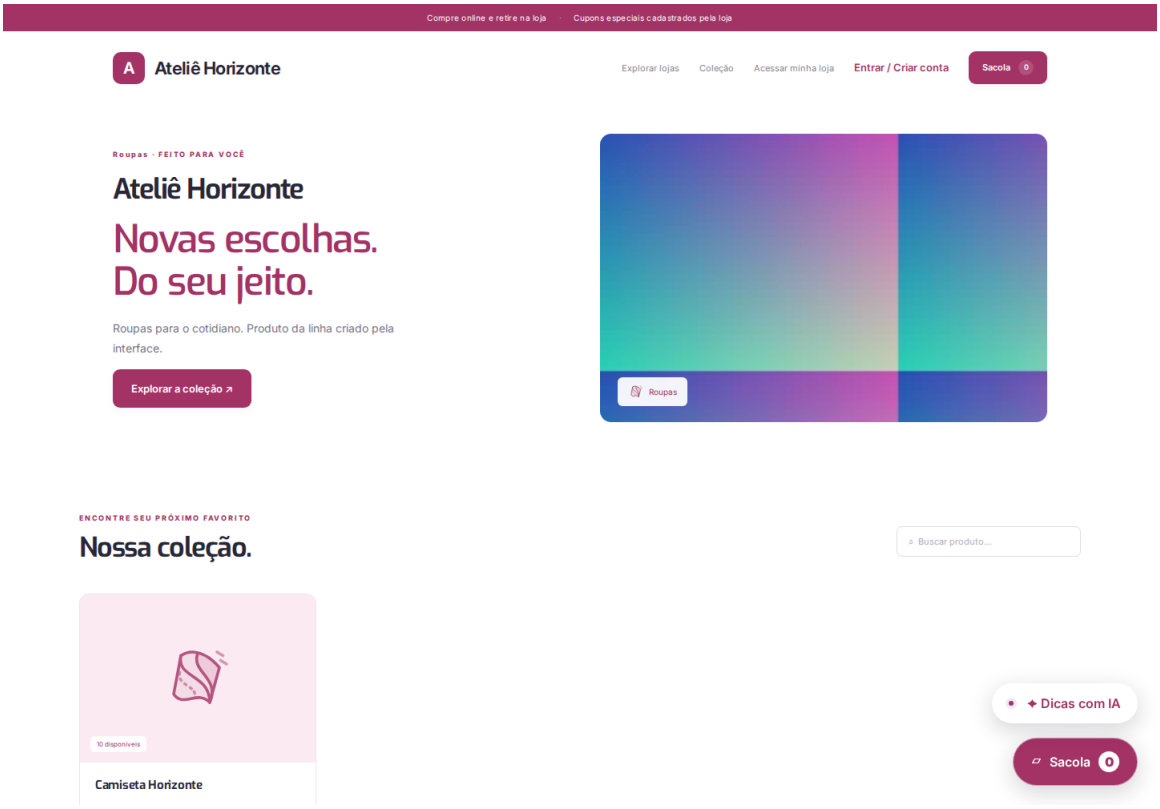


Figura 18 - Ateliê Horizonte publicado pela interface, com tema Roupas e acesso ao consultor opcional de IA.
Fonte: captura da aplicação com dados fictícios.

O produto inteligente reaproveita o núcleo e habilita `aiAssistant`. O chat público recebe catálogo e saldos e constrói cartões a partir de IDs validados. O agente administrativo pode consultar ferramentas de estoque e vendas. São fluxos diferentes das recomendações automáticas por histórico, que continuam disponíveis sem IA.

As capturas desta rodada não executaram Gemini real. A integração real do provedor está registrada nas evidências históricas de 09 e 10/11, com dados fictícios. Configuração presente não garante chave válida, quota ou disponibilidade externa. O sistema informa ausência de configuração e falhas do provedor em vez de simular uma resposta de chat.

8.5 Limites e evoluções

A revisão validada demonstra produtos selecionados da família, não todas as 48 combinações estruturais. O algoritmo de recomendação utiliza histórico limitado e sinais simples; não houve experimento de conversão ou avaliação com usuários. A categorização atual dos produtos pode reduzir a diferenciação do sinal de categoria dentro de uma loja.

Swagger tem detalhamento parcial de DTOs. Não há worker automático para reconciliar tentativas de compra abandonadas; a retomada depende da mesma referência. O banco compartilha instância e credencial, e o limitador por IP é local ao gateway. Não fazem parte da entrega pagamento real, recuperação de senha, emissão fiscal e configuração de produção.

9 Repositório e instruções de execução

O repositório completo está em https://github.com/CaioMascarenhas/gerar-commerce. O link aparece também na capa e no sumário. A revisão utilizada é https://github.com/CaioMascarenhas/gerar-commerce/tree/5435ce33c7c0cb0f5cd226938793e771464b64a9. O quadro está em https://github.com/users/CaioMascarenhas/projects/3/views/2. O acesso depende da permissão no repositório.

Ativo ou serviço	Código-fonte completo no GitHub
framework	backend/packages/framework/src
contracts	backend/packages/contracts/src
platform	backend/packages/platform/src
gateway	backend/services/gateway/src
stores	backend/services/stores/src
catalog	backend/services/catalog/src
inventory	backend/services/inventory/src
orders	backend/services/orders/src
recommendations	backend/services/recommendations/src
ai-assistant	backend/services/ai-assistant/src
frontend/src/app	frontend/src/app
compose.yaml	compose.yaml

9.1 Inicialização com Docker Compose

São necessários Git, Docker com motor ativo e Compose v2. O build em contêiner dispensa Node no host. Na raiz da cópia do projeto:

```
git clone https://github.com/CaioMascarenhas/gerar-commerce.git
cd gerar-commerce
docker compose up --build -d --wait
docker compose ps
```

A plataforma fica em `http://localhost:4200` e a saúde agregada em `http://localhost:3000/api/services`. O cadastro em `/criar` publica a loja. Não há necessidade de executar o seed para usar a plataforma; `npm run seed`, no backend, é uma opção para criar lojas demonstrativas. Dados existentes são preservados pelo volume.

`FRONTEND_PORT`, `GATEWAY_PORT` e `POSTGRES_PORT` ajustam portas publicadas. `docker compose logs -f recommendations` acompanha o serviço de ranking. `docker compose stop` e `docker compose start` preservam o banco. A porta 3004 é utilizada internamente por recomendações.

9.2 Configuração opcional de Gemini

O Compose lê `backend/.env` para o serviço de IA. `GOOGLE_API_KEY` e `GEMINI_MODEL` devem conter chave e modelo válidos. Selecionar `aiAssistant` habilita o recurso na loja; a configuração do provedor é uma condição operacional separada. Nenhuma credencial real faz parte do relatório ou do frontend.

```
# Após alterar somente as variáveis do serviço de IA:
docker compose up -d --force-recreate ai-assistant
```

Recriar o contêiner atualiza seu ambiente; não é necessário reconstruir a imagem apenas por esses valores. Sem Gemini, compra e recomendação por histórico continuam utilizáveis. A geração inicial oferece exemplos identificados, e os chats informam indisponibilidade.

9.3 Desenvolvimento e verificações

Para execução local, instalar Node no intervalo de `engines`, iniciar PostgreSQL e instalar dependências de backend e frontend separadamente. `npm run dev`, no backend, compila e inicia os sete processos. `npm start`, no frontend, inicia o Angular.

```
# Em backend/
npm ci
npm test
npm run build
npm run smoke

# Em frontend/
npm ci
npm test
npm run build
npm run test:e2e:types
npm run test:e2e
```

O smoke utiliza banco temporário; os E2E usam projeto Compose, rede e volume exclusivos, sem portas públicas. O runner remove somente seus recursos temporários. Para Gemini real, o modo opcional é `npm run test:e2e -- --ai`, com configuração válida e dados de demonstração.

10 Referências bibliográficas

[1] UNIVERSIDADE FEDERAL DE ALAGOAS. Elementos do Relatório do Projeto: Tópico em Engenharia de Software, Projetando Linhas de Produto de Software. Modelo 2026.2. Arquivo: projeto_docs/Elementos do Relatorio do Projeto -2026-2-lps.pdf.

[2] KANBAN GUIDES. The Kanban Guide. Maio de 2025. Disponível em: https://kanbanguides.org/the-kanban-guide/2025.5/. Acesso em: 10 nov. 2026.

[3] CONVENTIONAL COMMITS. Conventional Commits 1.0.0. Disponível em: https://www.conventionalcommits.org/pt-br/v1.0.0/. Acesso em: 10 nov. 2026.

[4] NORTHROP, Linda M. Software Product Lines. Software Engineering Institute, Carnegie Mellon University, 2005. Apresentação e resumo institucional. Disponível em: https://www.sei.cmu.edu/library/software-product-lines/. Acesso em: 10 nov. 2026.

[5] REFACTORING.GURU. Template Method. Disponível em: https://refactoring.guru/design-patterns/template-method. Acesso em: 10 nov. 2026.

[6] LEWIS, James; FOWLER, Martin. Microservices. 2014. Disponível em: https://martinfowler.com/articles/microservices.html. Acesso em: 10 nov. 2026.

[7] POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17 Documentation: Schemas. Disponível em: https://www.postgresql.org/docs/17/ddl-schemas.html. Acesso em: 10 nov. 2026.

[8] MASTRA. Tools. Documentação oficial. Disponível em: https://mastra.ai/docs/agents/tools. Acesso em: 10 nov. 2026.

[9] ANGULAR. What is Angular? Documentação oficial. Disponível em: https://angular.dev/overview. Acesso em: 10 nov. 2026.

[10] NESTJS. Documentation. Disponível em: https://docs.nestjs.com/. Acesso em: 10 nov. 2026.

[11] DOCKER. Why use Compose? Documentação oficial. Disponível em: https://docs.docker.com/compose/intro/features-uses/. Acesso em: 10 nov. 2026.

[12] SOARES, Caio Mascarenhas; COSTA, Laura Marques. Gerar Commerce. Código-fonte e documentação. Revisão 5435ce33c7c0cb0f5cd226938793e771464b64a9. Disponível em: https://github.com/CaioMascarenhas/gerar-commerce. Acesso em: 10 nov. 2026.

[13] UNIVERSIDADE FEDERAL DE ALAGOAS. Identidade Visual. Marca oficial utilizada na capa, com proporções preservadas. Disponível em: https://ufal.br/ufal/comunicacao/identidade-visual. Arquivo: https://ufal.br/ufal/comunicacao/identidade-visual/brasao/sigla/png.png. Acesso em: 10 nov. 2026.